

ТЕОРИЯ

<https://vk.com/@gofor1100-zadanie-4-kodirovanie-i-dekodirovanie-dannyh-uslovie-fano>

ТЕОРИЯ

[двоичное дерево.jpg](#)

[дерево фано.doc](#)

Есть несколько факторов, которые нужно соблюсти, чтобы кодирование прошло хорошо.

Но чтобы научиться делать что-то правильно, сначала надо разобраться, как сделать неправильно.

Давайте попробуем произвольным образом закодировать буквы К, У, Р, Л, Ы, чтобы составлять из них сообщения.

К	У	Р	Л	Ы
010	01	00	10	001

Каждой букве в табличке сопоставлено несколько символов, в нашем случае цифр. Эти символы — **кодированное слово** буквы, позволяющее отличить ее от других букв в закодированном сообщении.

С помощью **кодированных слов** каждой отдельной буквы закодируем сообщение КУРЛЫК, для этого на место каждой буквы будем подставлять соответствующее ей кодированное слово:

К.	У.	Р.	Л.	Ы.	К.
010.	01.	00.	10.	001.	010

С проблемой мы столкнемся при попытке расшифровать сообщение. Чтобы это сделать, надо выделить в коде сообщения отдельные кодированные слова и подставить на их место соответствующие буквы. Но при расшифровке этого кода можно получить другой вариант:

010001000100001010
УЫЫРКЛ

Как закодировать сообщение КУРЛЫК правильно?

Чтобы кодирование считалось качественным и надежным, код должен удовлетворять условию **однозначного декодирования** — после кодирования сообщений у них должен быть единственный вариант расшифровки.

РАВНОМЕРНОЕ КОДИРОВАНИЕ

Но как можно соблюдать это условие? Или еще круче — как строить кодированные слова, которые заранее будут этому условию удовлетворять?

Что такое равномерное кодирование?

Это первый вариант подхода к однозначному декодированию. Он подразумевает кодирование отдельных элементов кодами, имеющими одинаковую длину.

И такой подход действительно спасает ситуацию. Если каждое кодированное слово будет иметь одинаковую длину, и при этом все они будут различны, выделять в зашифрованном сообщении отдельные кодированные слова не составит труда, как и сопоставлять их с соответствующими им буквами.

К	У	Р	Л	Ы
000	001	010	011	100

Кодируем сообщение КУРЛЫК:

КУРЛЫК
000001010011000000

И при попытке декодирования, **поочередно выделяя кодированные слова одинаковой длины**, мы не имеем никаких проблем с сопоставлением и получаем исходное сообщение без альтернативных вариантов.

000.	001.	010.	011.	100.	000
К.	У.	Р.	Л.	Ы.	К

Такой метод действительно прост и в освоении, и при составлении кодированных слов, и при расшифровке.

Но есть один минус — **он не всегда эффективен**. Выбирая длину одного кодированного слова, мы можем заранее определить количество возможных кодов: так как на каждой позиции может быть либо 0, либо 1, количество возможных кодов N длины i будет равно $N = 2^i$.

Выбрав длину 3 для наших кодированных слов, мы имеем 8 различных кодов, хотя кодируем всего 5 букв.

В других ситуациях потери могут быть плачевнее: для кодирования, например, 33 букв русского алфавита длина кодового слова должна быть не меньше 6 (если выберем 5, получим максимум $2^5 = 32$ кодового слова, но тогда 1 буква останется незакодированной).

Но имея $2^6 = 64$ кодовых слова для 33-х букв алфавита, мы явно взяли кодов больше, чем нужно.

Почему это плохо? Информация занимает много места.

УСЛОВИЕ ФАНО

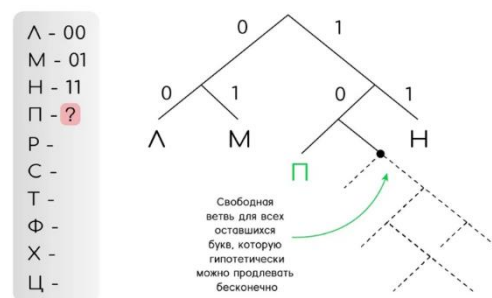
Хороший вариант — составление кодовых слов, которые удовлетворяют условию Фано.

УСЛОВИЕ ФАНО ГЛАСИТ:

«ни одно кодовое слово не должно быть НАЧАЛОМ другого кодового слова».

Такая формулировка **НЕ допустит** сосуществование кодовых слов 011 и 01, но **допустит** 110 и 10

Очень просто строить код, который будет удовлетворять условию Фано, с помощью **дерева вариантов** — структуры, в нашем случае помогающей получить кодовые слова максимально просто и понятно.

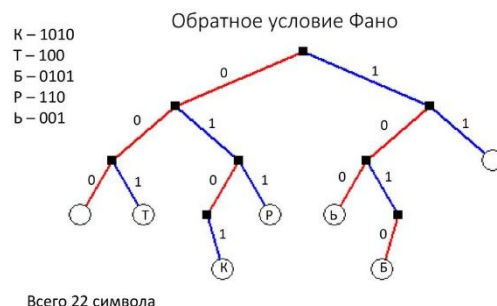


ОБРАТНОЕ УСЛОВИЕ ФАНО.

ОБРАТНОЕ УСЛОВИЕ ФАНО ГЛАСИТ:

«ни одно кодовое слово не должно быть ОКОНЧАНИЕМ другого».

Такая формулировка уже **допустит** сосуществование кодовых слов 011 и 01, но **НЕ допустит** 110 и 10



В жизни чаще всего используются **равномерные** кодировки в силу многих причин, таких как:

- **Большая устойчивость.**
- **Простота расширения.** (Проблема равномерного кодирования — в большом количестве «пустых мест». Но эти пустые места могут быть полезны. Например, захочет разработчик добавить в код несколько новых эмодзи. У него как раз есть незанятые коды для этих символов. Согласитесь, это удобнее, чем переписывать всю кодировку целиком.)

Неравномерные же кодировки часто применялись еще до появления компьютеров. Например, одной из самых известных неравномерных кодировок являлась **азбука Морзе**, применявшаяся в работе телеграфов, передаче радиосигналов и других способах передачи информации.